

HTTP Server Design Doc: David Glover

Modularization Plan:

The current plan is to split the code into 8 different files, each with a clear goal, that will hide the internals, and output only what other modules will require to carry out their specific functions.

Current file plans are main, server, http, connection, dispatcher, file, log, and parallel.

`main.c:`

This will be the file that sets up the program and parses the arguments. Its roles will be to validate the port number and determine whether the server needs to be parallelized. If it does this is where we will initialize the threadpool. This will also be where we initialize the server and keep it running until it is requested to be closed.

`server.c:`

In this file we will create the listening socket, bind to it, and begin to listen in a loop waiting for connections. In this file we accept connections and enqueue the requests into our thread pool.

This will be our networking module.

`http.c:`

In this file we will parse the http request line and headers. This is also where we will handle all errors in an http request, such as malformed line, inaccessible files etc. We won't perform any complex operations in this file other than parsing the input into its required parts.

connection.c:

We will use this file to handle one connection from end to end. It will receive the file descriptor for the client and use it to read from the TCP socket into a buffer. We will then pass that buffer to http.c to parse the request. We will determine whether this is a GET or a PUT request and pass that onto dispatcher.c. We will also call log.c to record the log of this connection. After the connection is complete, we will close the socket.

dispatcher.c:

This is the file where we will implement the GET/PUT semantics. With each specified file we will initiate and hold a lock to ensure no concurrency violations once we parallelize. For GET requests we will check that the file path exists, check our permissions, and if accessible, read the file. For PUT requests we will create and write to the file if it doesn't exist, and overwrite the file contents if it does. We can use a shared hashmap in this file to keep track of all the mutexes, if we make the filepath/name the key, we can simply check if the lock is being held, and if so block the thread trying to access it until the lock is released. This is how we will prevent data races when writing to files.

file.c:

We will use this file to securely open files with the specified flags, read the file contents into memory, and write the contents to the disk. This file will also have to detect whether or not the file exists and return error codes related to errors in the file specification

log.c:

We will use this file to produce the required audit logs to stderr. It will format the audit logs, write them to stderr. In this file we will implement a global lock that will prevent data races in the audit log and ensure that they are printed in the order they finished in.

parallel.c:

We will use this file only when the -p flag is specified, and will implement the system to concurrently call the required functions or methods. This is where we will implement our thread pool that creates the specified number of workers which listen until a connection is made. Using a threadpool will save us the overhead of creating a new thread for each request and ensure that files are accessed in the order they are requested.

Shared Structures

Job queue - bounded, circular; shared between listener and worker threads.

File lock table - maps filenames to mutexes.

Logging state - next sequence number to print, array of pending log records.

Parallelization:

Thread Pool Model

The system uses one listener thread and a pool of worker threads:

Listener thread

- Accepts new connections.

- Assigns a unique sequence number.

- Enqueues the new job into the shared job queue.

Worker threads

- Continuously dequeue jobs.

- Parse the HTTP request.

- Acquire the per-file lock for the target file.

- Execute GET or PUT.

Send the HTTP response back to the client.

Submit a log entry.

Why threads?

Lower overhead than processes.

Direct shared memory access simplifies the job queue, file lock table, and logging.

Thread pools avoid per-connection thread creation and destruction. We plan to ensure the state is synchronized correctly by assigning a number to each job and then using mutexes to make sure the audit logs are in the same order as the jobs are in.

Abstraction:

When dealing with the Abstraction section of our project, we plan to expose only a very minimal interface to the user. This is how the abstraction would be for each file:

1. server.c

Goal: Have an interface for start server, accept connections, and shutting down.

Hidden: socket creation, bind flags, error conversions.

Exposed API:

```
int server_init(int port);
```

```
int server_accept(int listening_fd);
```

```
void server_close(int fd);
```

2. connection.c

Goal: Manage a client connection from accept → read → parse → dispatch → respond
→ close.

Hidden: buffering raw bytes, calling http.c, invoking dispatcher.c, logging, closing sockets.

Exposed API:

```
void handle_connection(int client_fd);
```

3. http.c

Goal: To have a representation of parsed HTTP requests and responses.

Hidden: regex-like validation(strtok), malformed-line detection, header parsing, version checking, Content-Length logic.

Exposed API:

```
int http_parse_request(char *raw, http_request_t *req);
```

```
void http_build_error(http_response_t *resp, int code);
```

4. dispatcher.c

Goal: After getting a parsed request to see if it is a GET/PUT and then execute the correct handler further, also creating the hashmap of mutexes.

Hidden: GET/PUT semantics, return code selection, existence/permission checks, working-directory mapping.

Exposed API:

```
void dispatch_request(http_request_t *req, http_response_t *resp);
```

5. file.c

Goal: Have wrappers for open(), read(), write(), close().

Hidden: checking errno values, partial read/write loops, permission handling.

Exposed API:

```
int file_read_all(const char *path, uint8_t **buf, size_t *len);
```

```
int file_write_all(const char *path, uint8_t *data, size_t len);
```

```
int file_exists(const char *path);
```

6. log.c

Goal: Have a function that emits audit logs in a consistent format.

Hidden: formatting, writing to stderr, guarding formatting from malformed input.

Exposed API:

```
void log_request(http_request_t *req, int status_code);
```

7. parallel.c

Goal: Hide all the concurrency elements behind a queue-based system(like a queue-based server).

Hidden: worker creation, mutexes, condition variables, queue operations, job dispatch, joining.

Exposed API:

```
void parallel_init(int parallelism);
```

```
void parallel_submit(int client_fd);
```

```
void parallel_shutdown(void);
```

8. main.c

Goal: Tying together all the other files/functions while keeping the abstraction for the file

Hidden:

- All details of getopt parsing
- Converting strings to ints (port, parallelism)

- Printing the "Invalid Port\n" error logic
- Deciding whether to call parallel or sequential code paths
- Ensuring worker threads/processes are started or stopped correctly

Exposed API:

```
int parse_args(int argc, char **argv);
```

```
int initialize_system(int port, int parallelism);
```

```
void run_server_loop(void);
```

```
void shutdown_system(void);
```

Local Tests (outside autograder):

We will use the built in GitLabs CI pipeline to automate compilation and testing. This way we can define the tests that each file will need to pass, before even writing any server code. Since we have already planned our modularization and abstractions, we can write tests to ensure that the build completes and that each file correctly outputs its intended result before the entire system is complete.

I have created a test-config.txt file that we will use to keep track of which modules are a work in progress and which ones are finished in order to be able to continuously test as we go. WIP modules will have their corresponding test cases skipped, since they aren't properly implemented and the complete modules marked as DONE will run their allotted tests.

The CI/CD pipeline method of testing will be the most optimal way for us to locally check, since it will run the testing scripts automatically as we are working, letting us know if what we think is a complete module is functioning how we expect it, without having to manually run the tests.